```python
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# ============================================================
# QUESTION 1 — Sequence Input & Encoding
# ============================================================

sequence = input("Enter sequence (space separated words/numbers):
").split()

# Build vocabulary
vocab = list(set(sequence))
word_to_index = {w:i for i,w in enumerate(vocab)}

# One-hot encoding
def one_hot(idx, size):
    vec = np.zeros(size)
    vec[idx] = 1
    return vec

encoded_sequence = np.array(
    [one_hot(word_to_index[w], len(vocab)) for w in sequence]
)

print("\nEncoded Sequence:")
print(encoded_sequence)

# ============================================================
# Shared Parameters
# ============================================================

input_size = len(vocab)
hidden_size = 5
T = len(encoded_sequence)

# Random weights
Wx = np.random.randn(hidden_size, input_size)
Wh = np.random.randn(hidden_size, hidden_size)
```

```
b = np.zeros(hidden_size)

# ============================================================
# QUESTION 2 — RNN Hidden State Propagation
# ============================================================

h = np.zeros(hidden_size)
rnn_states = []

for x in encoded_sequence:
    h = np.tanh(Wx @ x + Wh @ h + b)
    rnn_states.append(h.copy())

rnn_states = np.array(rnn_states)

print("\nRNN Hidden States:")
print(rnn_states)

# ============================================================
# QUESTION 3 — Memory Decay Analysis
# ============================================================

# influence of first token
h = np.zeros(hidden_size)
memory = []

x0 = encoded_sequence[0]

for t in range(T):
    x = encoded_sequence[t]
    h = np.tanh(Wx @ x + Wh @ h)
    influence = np.linalg.norm((Wh @ h))
    memory.append(influence)

plt.figure()
plt.plot(memory, marker='o')
plt.title("RNN Memory Decay")
plt.xlabel("Time Step")
plt.ylabel("Influence Magnitude")
plt.show()
```

```
# ======================================================
# QUESTION 4 — LSTM Gate Simulation
# ======================================================

def sigmoid(x):
    return 1/(1+np.exp(-x))

Wf = np.random.randn(hidden_size, input_size)
Wi = np.random.randn(hidden_size, input_size)
Wo = np.random.randn(hidden_size, input_size)
Wc = np.random.randn(hidden_size, input_size)

hf = np.zeros(hidden_size)
c = np.zeros(hidden_size)

lstm_memory = []

for x in encoded_sequence:

    f = sigmoid(Wf @ x)
    i = sigmoid(Wi @ x)
    o = sigmoid(Wo @ x)
    c_bar = np.tanh(Wc @ x)

    c = f*c + i*c_bar
    hf = o*np.tanh(c)

    lstm_memory.append(np.linalg.norm(c))

lstm_memory = np.array(lstm_memory)

# ======================================================
# QUESTION 5 — RNN vs LSTM Memory Comparison
# ======================================================

plt.figure()
plt.plot(memory, label="RNN")
plt.plot(lstm_memory, label="LSTM")
plt.title("RNN vs LSTM Memory Retention")
```

```
plt.legend()
plt.show()

# ======================================================
# QUESTION 6 — GRU Gate Simulation
# ======================================================

Wr = np.random.randn(hidden_size, input_size)
Wz = np.random.randn(hidden_size, input_size)
Whg = np.random.randn(hidden_size, input_size)

hg = np.zeros(hidden_size)
gru_memory = []

for x in encoded_sequence:

    r = sigmoid(Wr @ x)
    z = sigmoid(Wz @ x)

    h_candidate = np.tanh(Whg @ x)
    hg = (1-z)*hg + z*h_candidate

    gru_memory.append(np.linalg.norm(hg))

gru_memory = np.array(gru_memory)

# ======================================================
# QUESTION 7 — LSTM vs GRU Comparison
# ======================================================

plt.figure()
plt.plot(lstm_memory, label="LSTM")
plt.plot(gru_memory, label="GRU")
plt.title("LSTM vs GRU Memory")
plt.legend()
plt.show()

print("\nObservation:")
print("LSTM stores longer memory via cell state.")
print("GRU is simpler and computationally efficient.")
```

```
# ============================================================
# QUESTION 8 — Unified Memory Visualization
# ============================================================

plt.figure()
plt.plot(memory, label="RNN")
plt.plot(lstm_memory, label="LSTM")
plt.plot(gru_memory, label="GRU")

plt.title("Unified Memory Comparison")
plt.xlabel("Time Step")
plt.ylabel("Memory Strength")
plt.legend()
plt.show()

# ============================================================
# QUESTION 9 — Sequence Prediction
# ============================================================

def predict_next(state):
    similarity = encoded_sequence @ Wx.T
    scores = similarity @ state
    return vocab[np.argmax(scores)]

rnn_pred = predict_next(rnn_states[-1])
lstm_pred = predict_next(hf)
gru_pred = predict_next(hg)

print("\nNext Element Prediction:")
print("RNN Prediction :", rnn_pred)
print("LSTM Prediction:", lstm_pred)
print("GRU Prediction :", gru_pred)

# ============================================================
# QUESTION 10 — Conceptual Reflection
# ============================================================

reflection = """
```

```
RNNs fail on long sequences because gradients shrink during
backpropagation,
causing earlier information to vanish. This is known as the vanishing
gradient problem.
LSTM introduces gates and a dedicated cell state that allows information
to flow
unchanged across many time steps. GRU simplifies this idea using update
and reset gates
while maintaining long-term dependencies efficiently.
Because sequential recurrence still limits parallel computation,
Attention mechanisms were introduced to directly connect distant tokens.
This eventually led to Transformers, which model global dependencies
without relying on recurrence.
"""

print("\nConceptual Reflection:")
print(reflection)
```

```
PS C:\Users\vaibh\OneDrive\Desktop\ga> python ga4.py
Enter sequence (space separated words/numbers): vaibhav singh

Encoded Sequence:
[[1. 0.]
 [0. 1.]]

RNN Hidden States:
[[ 0.45952909  0.57011185 -0.22996582  0.91847887 -0.43777453]
 [-0.93215224  0.3792804  -0.6499198  -0.33474605 -0.00640627]]
```

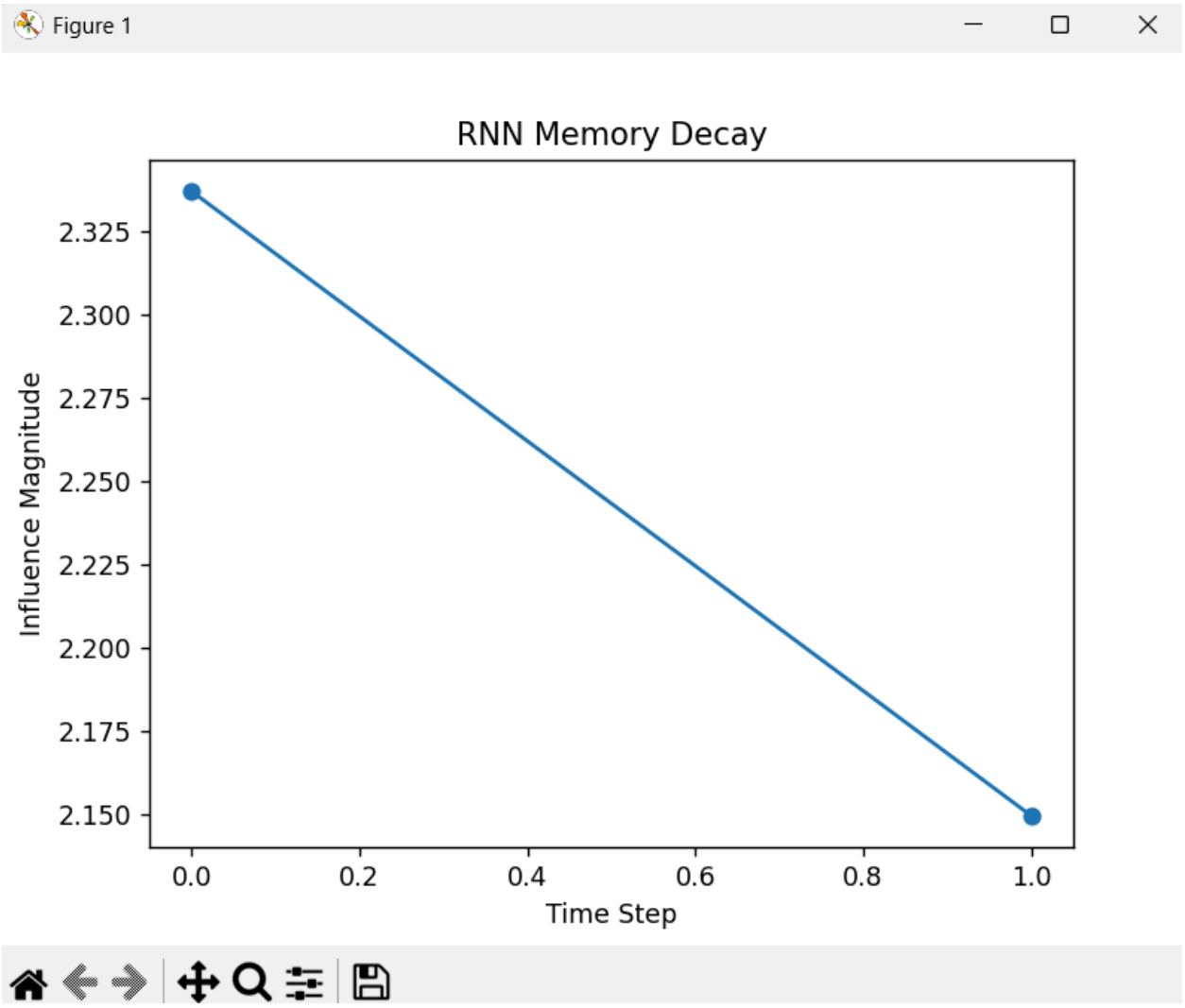
Figure 1
RNN Memory Decay
Influence Magnitude
2.325
2.300
2.275
2.250
2.225
2.200
2.175
2.150
0.0
0.2
0.4
0.6
0.8
1.0
Time Step